CO2 ASSESSMENT TOOL 2 — ARCHITECTURE DESIGN ASSIGNMENT

SIMATS ENGINEERING

(Saveetha School of Engineering)

Department of Computer Science and Engineering

STUDENT NAME	NITHIYASRI S
REGISTER NUMBER	192572082
COURSE CODE	CSA1017
COURSE NAME	Software Engineering
CO ASSESSED	CO2 — Requirements analysis, scalable architecture design & feature prioritization

Table of Contents

Architecture Design Analysis

This report presents a comprehensive architecture design analysis for the AI-Powered Smart Vocational Skill Assessment and Career Recommendation Platform, prepared in accordance with the requirements of CO2 Assessment Tool 2 of the Software Engineering course. The objective of this assignment is to analyze the assigned problem statement, design a scalable and reliable software architecture, and justify the architectural decisions taken while addressing the essential quality attributes expected of a modern, industry-grade software system.

1. Industry Context and Problem Overview

Educational institutions, vocational training organizations, and industries today face a persistent challenge in matching the skills possessed by learners with the rapidly evolving requirements of the job market. Traditional career counselling approaches are largely manual, generic, and disconnected from real-time labour market intelligence, resulting in learners choosing career paths that may not align with their actual aptitude, interests, or employability potential. At the same time, industries struggle to locate candidates who possess the precise blend of technical and soft skills that a given role demands.

Advances in artificial intelligence, natural language processing, data analytics, and cloud-based assessment platforms have created an opportunity to bridge this gap. An AI-powered career recommendation platform can objectively evaluate a learner's technical competencies, behavioural and soft skills, aptitude, and learning preferences, and then computationally match this profile against current industry skill demands. Based on this matching process, the platform can recommend personalized career pathways, relevant certification and training programs, and potential job opportunities, while continuously monitoring the learner's progress over time.

The proposed system is therefore expected to function as a continuous, data-driven bridge between education and employment — assessing competencies, benchmarking them against industry expectations, and guiding learners toward outcomes that improve both individual employability and overall workforce readiness.

Core Objectives of the System

- Assess technical and soft skills of students and job seekers through structured, AI-assisted evaluations.
- Analyze aptitude and cognitive/learning preferences using machine learning models.
- Recommend suitable career paths based on the learner's evaluated profile.
- Match individual skill sets with real-time industry demand data.
- Suggest relevant training and certification programs to close identified skill gaps.
- Generate personalized, easy-to-understand career reports for learners and institutions.
- Continuously monitor learning progress and re-evaluate recommendations over time.
- Improve overall employability and workforce readiness of the learner population.

2. Analyze System Requirements

A careful requirements analysis is the foundation of any sound architectural decision. For the AI-Powered Smart Vocational Skill Assessment and Career Recommendation Platform, the requirements were derived directly from the problem statement, the stated objectives, and the expected outcomes, and were then classified into functional requirements, non-functional requirements, and the quality attributes that most strongly influence the choice of architecture.

2.1 System Objectives

The system objectives, as identified from the problem statement, are to provide an intelligent, scalable, and personalized platform that evaluates a learner's technical and soft skills, analyzes aptitude and learning style, benchmarks the resulting profile against live industry skill demand, and produces actionable, personalized career and training recommendations. The system must additionally track learner progress over time and support a growing base of students, institutions, and industry partners without degradation in performance.

2.2 Functional Requirements

Functional requirements describe the specific behaviours and capabilities that the platform must provide to its users. The key functional requirements identified for this system are as follows:

- User registration and profile management for students, institutions, and recruiters, including secure authentication and role-based access control.
- An online skill-assessment module capable of administering technical assessments (coding tests, subject quizzes, practical simulations) and soft-skill assessments (communication, teamwork, problem-solving).
- An aptitude and learning-preference module that captures cognitive strengths and preferred learning styles through psychometric-style questionnaires.
- An AI/ML-based analytics engine that processes assessment responses using natural language processing and machine learning models to derive a structured competency profile.
- A career-matching engine that compares the learner's competency profile against a continuously updated industry skill-demand repository and recommends suitable career paths.
- A recommendation module that suggests relevant certification courses, training programs, and skill-development resources aligned with identified gaps.
- A report-generation module that compiles a personalized, easy-to-read career report summarizing strengths, gaps, and recommended pathways.
- A progress-monitoring module that tracks a learner's skill development over multiple assessment cycles and updates recommendations accordingly.
- Notification services to inform learners, institutions, and advisors about assessment results, new recommendations, and relevant opportunities.
- Administrative dashboards for institutions and recruiters to view aggregated skill trends, shortlist candidates, and monitor placement-readiness metrics.

2.3 Non-Functional Requirements

Non-functional requirements define the quality and operational characteristics that the system must satisfy while delivering its functional capabilities. These requirements have a direct and significant influence on the choice of architecture.

- **Scalability** — the platform must support a growing number of concurrent learners, institutions, and recruiters, including seasonal spikes during admission and placement cycles, without performance degradation.
- **Performance** — assessment submissions and AI-driven analysis must be processed with minimal latency so that learners receive near real-time feedback.
- **Reliability** — the platform must consistently deliver correct assessment results and recommendations, since inaccurate outputs could adversely affect a learner's career decisions.
- **Availability** — the system must remain accessible with minimal downtime, particularly during peak assessment and placement windows.
- **Security** — sensitive personal, academic, and psychometric data must be protected through encryption, secure authentication, and strict access control, in line with data-protection expectations.
- **Maintainability** — individual modules (assessment, AI engine, career matching, reporting) must be independently upgradable as assessment techniques and industry data evolve.
- **Interoperability** — the platform must integrate smoothly with external job portals, certification providers, and industry skill-demand APIs.
- **Usability** — the user interface must be simple and accessible for students with varying levels of digital literacy, across both web and mobile channels.
- **Auditability** — all assessment and recommendation events should be logged for transparency, quality assurance, and continuous improvement of the AI models.

2.4 Key Quality Attributes Influencing the Architecture

Among the non-functional requirements, six quality attributes emerge as the most influential in shaping the architectural style of the platform: scalability, maintainability, reliability, availability, performance, and security. Because the platform must serve a growing and geographically distributed user base consisting of students, training institutions, and recruiters, and must integrate computationally intensive AI/ML processing with multiple external systems, an architecture that supports independent scaling of individual functions, fault isolation, and secure, well-defined integration points is essential. These considerations directly motivate the architectural style selected in the next section.

3. Select a Suitable Software Architecture

3.1 Candidate Architectural Styles Considered

Before finalizing an architectural style, several candidate styles were evaluated against the requirements identified above.

- **Monolithic Layered Architecture** — simple to build initially, but tightly couples the assessment engine, AI/ML processing, and reporting logic, making independent scaling and technology upgrades difficult.
- **Client-Server Architecture** — provides basic separation between clients and a central server, but does not, on its own, provide the fine-grained scalability or fault isolation required for a computationally intensive AI-driven platform.
- **Model-View-Controller (MVC)** — well suited to organizing a single application's presentation logic, but insufficient by itself to address distributed scalability and independent service evolution.
- **Microservices Architecture** — decomposes the system into independently deployable, loosely coupled services, each responsible for a specific business capability, and communicating through well-defined APIs and asynchronous messaging.
- **Event-Driven Architecture** — enables asynchronous, non-blocking communication between components using events, which is valuable for long-running AI/ML analysis tasks.
- **Service-Oriented Architecture (SOA)** — shares similarities with microservices but typically relies on heavier enterprise service buses and coarser-grained services.

3.2 Selected Architecture: Hybrid Microservices with Event-Driven Communication

- The system follows a **Hybrid Microservices Architecture with Event-Driven Communication** to achieve scalability, flexibility, and efficient processing.
- Business functionalities are divided into independent microservices such as User Management, Skill Assessment, AI Analytics, Career Recommendation, Notification, and Progress Tracking.
- An **API Gateway** serves as the central access point, routing requests to the appropriate services.
- Synchronous communication is used for real-time operations such as login, assessment submission, and profile management.
- Asynchronous communication through an **Event Bus/Message Broker** is used for AI analysis, recommendation generation, notifications, and report creation.
- The architecture supports modular development, allowing each service to be developed, tested, and deployed independently.
- A layered structure consisting of Presentation Layer, Business Services Layer, and Data Layer improves system organization and maintainability.

- The architecture enables seamless integration of AI and Machine Learning models without affecting other system components.
- Independent scaling of services helps handle increasing numbers of users and assessment requests efficiently.
- Event-driven communication reduces service dependency and improves overall system responsiveness.
- Fault isolation ensures that failures in one service do not impact the entire platform.
- The architecture supports future enhancements such as advanced analytics, additional recommendation modules, and third-party integrations.

3.3 Justification for the Selected Architecture

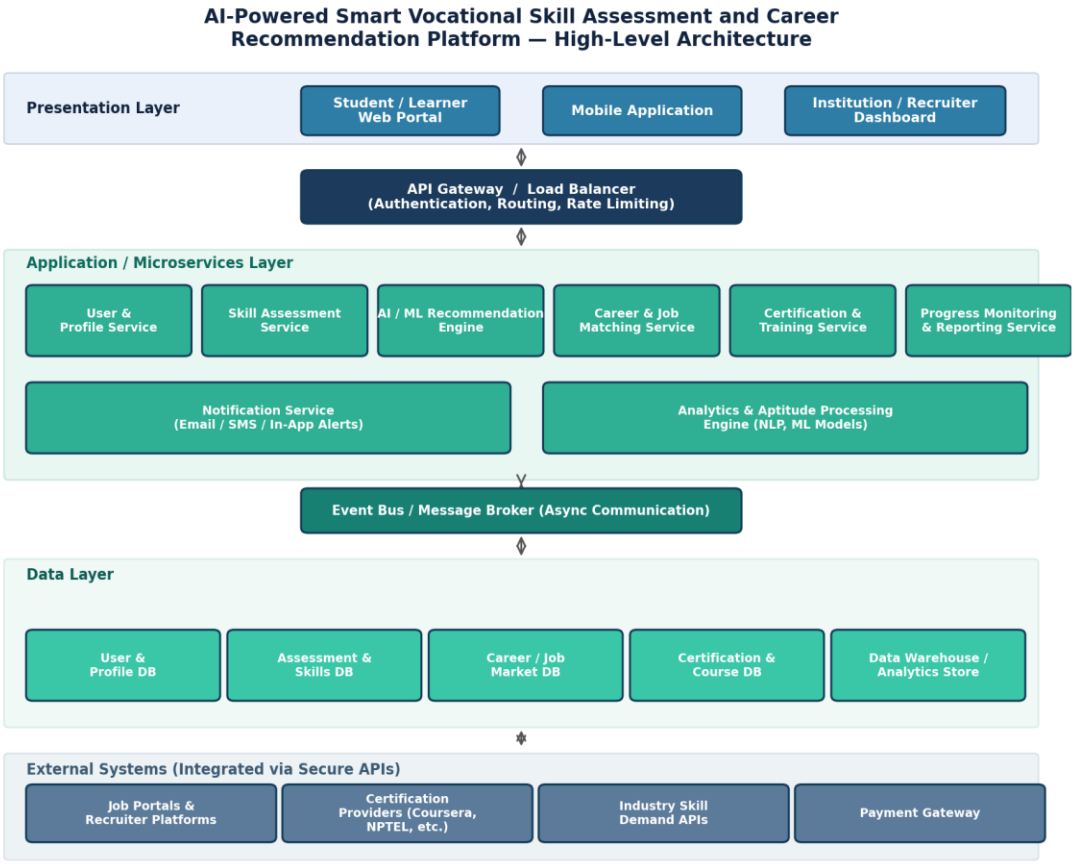
The hybrid microservices and event-driven architecture is the most suitable choice for this platform for the following reasons:

- **Independent Scalability** — the AI/ML recommendation engine is typically the most computationally demanding component; as a separate microservice, it can be scaled independently of lighter-weight services such as user management, without over-provisioning the entire system.
- **Fault Isolation and Reliability** — if one service (for example, the notification service) experiences a fault, the core assessment and recommendation functionality can continue operating, improving overall system reliability.
- **Technology Flexibility** — the AI/ML engine can be implemented using specialized data-science frameworks, while other services can use technologies best suited to their function, without forcing a single technology stack across the whole system.
- **Asynchronous Processing for AI Workloads** — skill assessment analysis and career matching can be time-consuming; an event-driven communication layer allows these operations to be processed asynchronously without blocking the user interface, improving perceived performance.
- **Ease of Integration** — well-defined APIs and an API gateway make it straightforward to integrate external systems such as job portals, certification providers, and industry skill-demand feeds.
- **Maintainability and Continuous Improvement** — because AI/ML models require frequent retraining and improvement, isolating the recommendation engine as its own service allows it to be updated and redeployed independently of the rest of the platform.
- **Support for Future Growth** — new capabilities, such as additional assessment types or new industry integrations, can be added as new microservices without redesigning the entire system.

Taken together, these characteristics make the hybrid microservices and event-driven architecture significantly better suited to this problem than a monolithic, purely layered, or purely client-server alternative, particularly given the platform's need to combine high-volume transactional operations with computationally intensive, asynchronous AI/ML processing.

4. Draw the Software Architecture Diagram

The overall structure of the proposed system is illustrated below. The diagram presents the major layers of the platform — the presentation layer, the API gateway, the application/microservices layer, the event bus, the data layer, and the external systems integrated through secure APIs — along with the primary communication mechanisms that connect them.



4.1 Description of the Architecture Diagram

- The **Presentation Layer** consists of the Student/Learner Web Portal, Mobile Application, and Institution/Recruiter Dashboard, providing interfaces for all users to access the platform. User requests are routed through a centralized **API Gateway and Load Balancer**, which manages authentication, authorization, request routing, and traffic distribution.

- The **Application/Microservices Layer** forms the core of the platform and contains independently deployable services for various business functions. Key microservices include User & Profile Management, Skill Assessment, AI/ML Recommendation, Career & Job Matching, Certification & Training Management, Progress Monitoring, Reporting, Notifications, and Analytics Processing.

- The **AI/ML Analytics Engine** analyzes assessment results and generates personalized career recommendations based on user skills, interests, and performance. Services requiring immediate responses communicate through synchronous API calls.

- Background processes such as AI analysis, notification delivery, and report generation use asynchronous communication through the **Event Bus / Message Broker**. The event-driven approach reduces service dependency and improves system scalability and reliability.

- The **Data Layer** stores information in dedicated databases for user profiles, assessments, career data, certifications, and analytics. A centralized analytics repository supports data analysis, reporting, and machine-learning model training.

- The **External Systems Layer** integrates third-party platforms such as job portals, recruiter systems, certification providers, industry skill-demand APIs, and payment gateways. Secure APIs ensure reliable communication between the platform and external services.

- The overall architecture enables efficient processing, high scalability, fault tolerance, and future expansion of platform features.

5. Explain Components and Their Interactions

5.1 Purpose and Responsibilities of Each Component

Each component in the architecture has a clearly defined responsibility, which keeps the overall system modular and easier to reason about.

- **User & Profile Service** — manages registration, authentication, role-based access, and profile data for students, institutions, and recruiters.
- **Skill Assessment Service** — administers technical and soft-skill assessments, captures learner responses, and stores raw assessment data.
- **AI/ML Recommendation Engine** — processes assessment and aptitude data using natural-language processing and machine-learning models to build a structured competency profile and generate ranked recommendations.
- **Career & Job Matching Service** — compares a learner's competency profile against the industry skill-demand repository and identifies suitable career paths and job opportunities.
- **Certification & Training Service** — recommends relevant certification courses and training programs based on identified skill gaps, and coordinates with external certification providers.
- **Progress Monitoring & Reporting Service** — tracks a learner's development across multiple assessment cycles and compiles the personalized career report.
- **Notification Service** — dispatches email, SMS, and in-app alerts to learners, advisors, and institutions regarding results, recommendations, and opportunities.
- **Analytics & Aptitude Processing Engine** — supports the AI/ML engine with aptitude analysis, learning-preference detection, and longer-term trend analytics.
- **Event Bus / Message Broker** — enables asynchronous, decoupled communication between services, particularly for long-running AI processing and notification tasks.

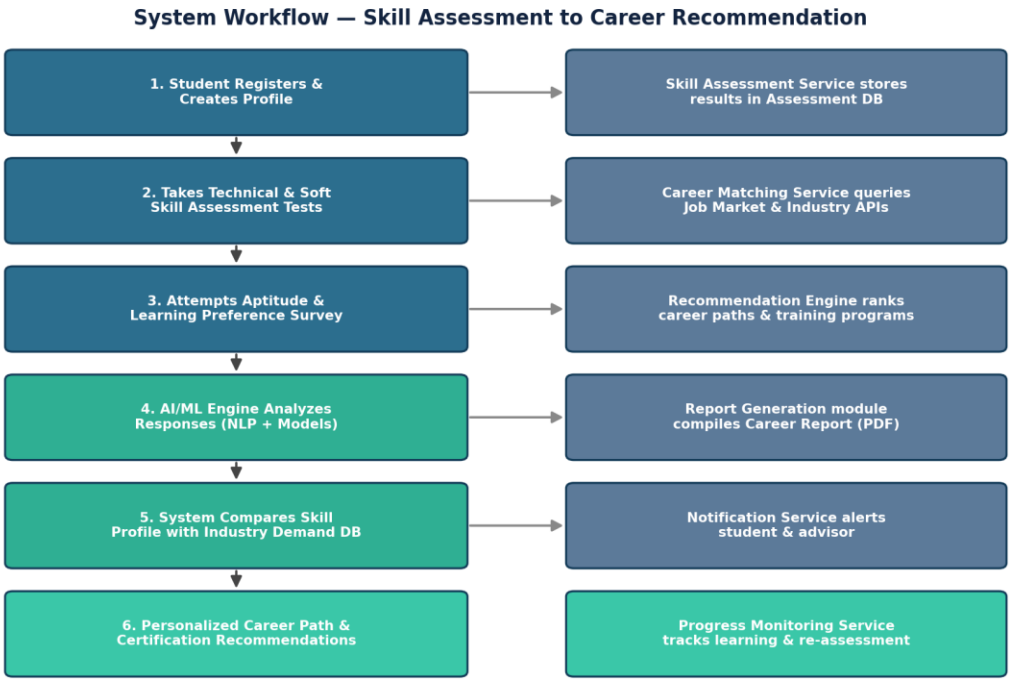
5.2 Communication and Data Exchange

Client applications communicate with the backend exclusively through the API Gateway, which authenticates each request and routes it to the corresponding microservice over lightweight, synchronous REST/HTTPS calls. For operations that are not time-critical from the user's perspective — such as AI/ML-based analysis of assessment responses, career-matching computations, notification dispatch, and report compilation — services publish events to the Event Bus rather than blocking on a direct call. Interested downstream services subscribe to these events and process them independently, which keeps the user interface responsive even while computationally heavy AI processing continues in the background.

Each microservice owns and manages its own data store within the data layer, following the principle of decentralized data management common to microservices architectures; services that need data owned by another service request it through that service's API rather than accessing the database directly, preserving loose coupling. External integrations — job portals, certification providers, industry skill-demand feeds, and the payment gateway — are accessed through dedicated adapter interfaces at the edge of the system, so that changes to any external provider's API affect only the corresponding adapter rather than the core platform.

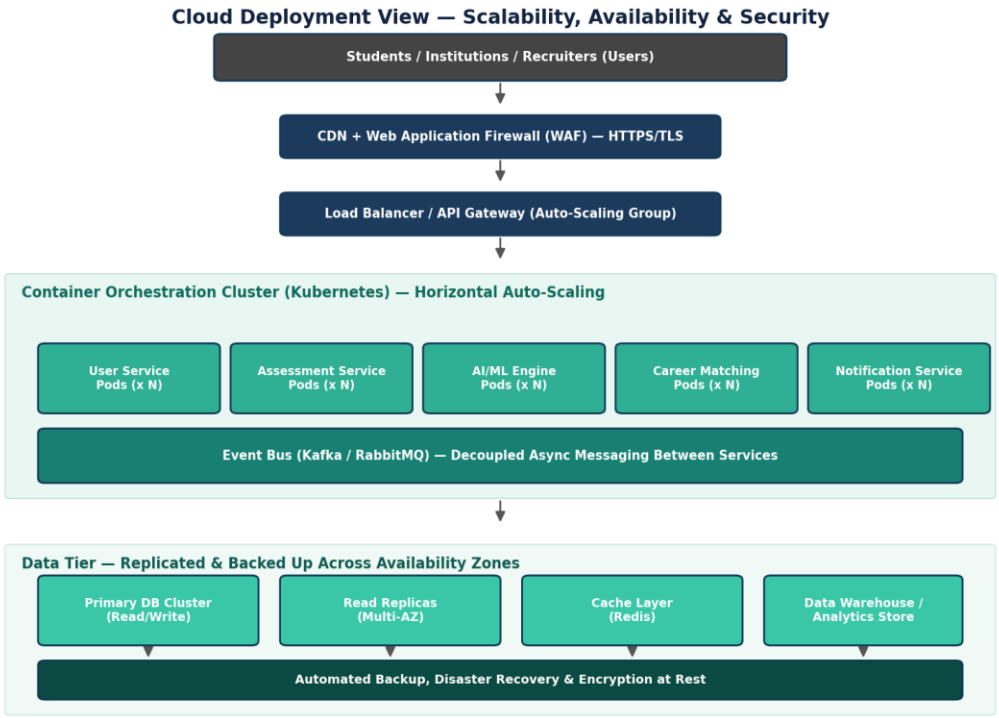
5.3 Overall System Workflow

The workflow diagram below illustrates how a learner's journey through the platform triggers the corresponding backend processing, from registration through to the delivery of a personalized career recommendation and its ongoing monitoring.



6. Discuss Quality Attributes

The proposed hybrid microservices and event-driven architecture was deliberately selected to satisfy the six quality attributes most critical to this platform. The cloud deployment view below illustrates the infrastructure-level mechanisms — auto-scaling, replication, caching, and security controls — that support these attributes in practice.



6.1 Scalability

Because each microservice is independently deployable, the platform can scale individual components in response to demand rather than scaling the entire system uniformly. The AI/ML Recommendation Engine and Skill Assessment Service, which typically experience the highest load during peak admission and placement seasons, can be horizontally scaled within the container-orchestration cluster (for example, using Kubernetes) by increasing the number of running pod instances, while lighter services such as notifications remain at baseline capacity. The event-driven communication layer further absorbs sudden bursts of assessment submissions by queuing them for asynchronous processing rather than overwhelming any single component.

6.2 Security

All external traffic passes through a Web Application Firewall and is encrypted end-to-end using HTTPS/TLS. The API Gateway centralizes authentication and authorization, applying role-based access control so that students, institutions, and recruiters see only the data and functionality relevant to their role. Sensitive data — including psychometric assessment results and personal profile information — is encrypted at rest within the data layer, and all access to this data is logged to support auditability and compliance with data-protection expectations.

6.3 Performance

The separation of synchronous and asynchronous communication paths allows the platform to keep user-facing interactions, such as viewing a dashboard or submitting an assessment, fast and responsive, while computationally intensive AI/ML analysis proceeds asynchronously in the background. A caching layer (for example, Redis) reduces repeated load on the primary databases for frequently accessed data such as career and certification catalogues, further improving response times.

6.4 Reliability

Fault isolation between microservices means that a failure or slowdown in one component — for instance, a temporary issue with an external certification-provider API — does not cascade into a failure of the entire platform. The event bus provides message durability, so that assessment or recommendation events are not lost even if a downstream consumer service is temporarily unavailable, and can be reprocessed once the service recovers.

6.5 Maintainability

Because each service encapsulates a single business capability with a well-defined API, individual services — most notably the AI/ML Recommendation Engine, whose underlying models will need frequent retraining and refinement — can be updated, tested, and redeployed independently, without requiring changes to unrelated parts of the system. This significantly reduces the risk and effort associated with ongoing enhancement of the platform.

6.6 Availability

The use of load balancing, auto-scaling groups, multi-availability-zone database replication, and automated backup and disaster-recovery mechanisms ensures that the platform remains accessible even in the event of individual node or service failures. Read replicas and cached data further allow read-heavy operations, such as browsing recommended career paths, to continue serving users even if the primary database experiences a temporary write-side issue.

7. Justify Architectural Decisions

7.1 Rationale Behind Key Architectural Choices

The decision to adopt a hybrid microservices and event-driven architecture, rather than a simpler monolithic or purely layered design, was driven primarily by the dual nature of the platform's workload: a high volume of relatively lightweight transactional operations (registration, profile updates, browsing recommendations) combined with a smaller number of computationally intensive, latency-tolerant AI/ML operations (skill analysis, aptitude modelling, career matching). Separating these concerns into independently scalable services, connected through both synchronous APIs and an asynchronous event bus, allows the platform to allocate computing resources efficiently and to keep the user experience responsive even while heavier analytical work is in progress.

The introduction of an API Gateway centralizes cross-cutting concerns such as authentication, routing, and rate limiting, avoiding the need to duplicate this logic across every service. Decentralizing data ownership, so that each service manages its own data store, was chosen to preserve loose coupling and to allow the schema and technology of each data store to evolve independently as requirements change — for example, the AI/ML engine's analytics store may eventually require a specialized format that differs from the transactional user database.

7.2 Trade-offs Considered

This architectural style introduces additional operational complexity compared to a monolithic design: multiple services must be deployed, monitored, and coordinated, and distributed communication introduces the possibility of partial failures that a single-process application would not encounter. Ensuring data consistency across service boundaries also requires more careful design than a monolithic system with a single shared database. These trade-offs were considered acceptable given the platform's expected growth, the need for independent scaling of the AI/ML workload, and the long-term maintainability benefits of a modular design. Where strict, immediate consistency is required — such as during assessment submission — synchronous API calls are used, while asynchronous, eventually-consistent processing is reserved for operations, such as career-recommendation refresh, where a short delay is acceptable.

7.3 Support for Future Enhancements and System Integration

The chosen architecture is well positioned to support future enhancements. New assessment types, additional AI/ML models (for example, more advanced natural-language interview simulators), or new industry integrations can be introduced as new microservices without requiring changes to existing, unrelated components. The API Gateway and adapter-based approach to external integration make it straightforward to onboard additional job portals, certification providers, or government skill-mission platforms as partners over time. Because the AI/ML Recommendation Engine is isolated as its own service, its models can be retrained and improved continuously — incorporating new industry-demand data or improved NLP techniques — without disrupting the rest of the platform.

8. Conclusion

The AI-Powered Smart Vocational Skill Assessment and Career Recommendation Platform addresses a genuine and pressing need to connect learner competencies with real-world industry demand through personalized, data-driven guidance. A careful analysis of the system's functional and non-functional requirements revealed that scalability, reliability, performance, security, availability, and maintainability are the quality attributes most critical to its success, given the combination of high-volume transactional usage and computationally intensive AI/ML processing that the platform must support.

A hybrid architecture combining microservices with event-driven communication, supported by a centralized API gateway and a resilient, replicated data and deployment infrastructure, was selected and justified as the most suitable architectural style for this system. This design enables independent scaling of the most demanding components, isolates faults to prevent cascading failures, keeps the platform responsive despite heavy background AI processing, and remains flexible enough to accommodate new assessment types, AI capabilities, and industry integrations as the platform evolves. Overall, the proposed architecture provides a strong, extensible foundation capable of delivering on the platform's objective of bridging the gap between education and employment through intelligent, personalized career guidance.